Lab 14 — Malware Static Analysis & Linux IOC Hunting

> **SY0-701 Objective 2.4** — Given a scenario, analyze indicators of malicious activity
>
> **Domain 2** — Threats, Vulnerabilities, and Mitigations (22%) · **Time:** ~30 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: triage a *harmless* test sample the way a SOC analyst triages real malware, then hunt classic malware indicators on a Linux host.

You'll build: a static-analysis workflow that produces a hash IOC and a YARA detection, plus a short Linux IOC-hunting checklist (suspicious process, cron persistence, unusual listening port).

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in step 1. The only "malware" used is the industry-standard EICAR test string, which is completely benign.

Background

- **Static analysis** inspects a sample *without running it* (file type, strings, hashes, YARA); **dynamic analysis** detonates it in a sandbox to watch behaviour.
- An **indicator of compromise (IOC)** is an observable artefact of an attack — a file hash, a domain, a mutex, a suspicious process or a persistence entry.
- **SHA-256** is the preferred IOC hash; MD5 is still seen in feeds but is collision-broken.
- **YARA** rules match text or byte patterns and are the standard way to turn an observation into a reusable detection.
- On Linux, common persistence and malware indicators include unexpected **processes**, **cron jobs**, and **listening ports** — this maps to objective 2.4's malware and indicator sub-bullets.

Steps

1. Install the analysis tools

```
apt-get update -qq && apt-get install -y yara binutils file procps iproute2 coreutils
```

2. Create the safe EICAR test sample

The EICAR string is a 68-byte ASCII file agreed by antivirus vendors so you can test detection without real malware.

```
printf '%s' 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' >
sample.txt
wc -c sample.txt
```

3. Identify the file type and compute the IOC hash

`file` reads magic bytes, not the extension; `sha256sum` produces the indicator you would submit to a threat feed.

```
file sample.txt
sha256sum sample.txt
```

4. Pull printable strings

Strings recover URLs, markers and ransom notes embedded in a binary — here, the EICAR marker itself.

```
strings sample.txt
```

5. Write a YARA rule and scan

A YARA rule has a `strings` block of patterns and a `condition`. This one matches the EICAR marker.

```
cat > eicar.yar <<'EOF'
rule EICAR_Test_File
{
    meta:
        description = "Detects the benign EICAR antivirus test string"
    strings:
        $marker = "EICAR-STANDARD-ANTIVIRUS-TEST-FILE"
    condition:
        $marker
}
EOF
yara eicar.yar sample.txt
```

```
EICAR_Test_File sample.txt
```

6. Simulate and hunt a suspicious process

Start a harmless long-running process to stand in for a beacon, then hunt for it the way you would during triage.

```
nohup sleep 6000 >/dev/null 2>&1 &
ps -eo pid,etimes,cmd --sort=-etimes | grep -v grep | grep 'sleep 6000'
echo "--- top CPU/mem processes ---"
ps aux --sort=-%cpu | head -5
```

7. Hunt cron persistence

Malware commonly persists through cron. Enumerate every cron location an attacker might use.

```
echo '*/5 * * * * root /tmp/.beacon.sh' > /etc/cron.d/suspicious
ls -la /etc/cron.d /etc/cron.daily 2>/dev/null
grep -R --include='*' . /etc/cron.d 2>/dev/null
crontab -l 2>/dev/null || echo "no user crontab"
```

8. Hunt unusual listening ports

A new or unexpected listening socket is a strong IOC for a backdoor or C2 agent.

```
ss -tulnp | tail -n +1
echo "--- flag anything not in the expected baseline (22/53/80/443) ---"
ss -tlnp | awk 'NR>1 {print $4}'
```

Test it

Confirm the static IOCs and the hunt findings.

```
sha256sum sample.txt | tee ioc.txt
yara eicar.yar sample.txt && echo "YARA MATCH OK"
pgrep -af 'sleep 6000' >/dev/null && echo "SUSPICIOUS PROCESS FOUND"
test -f /etc/cron.d/suspicious && echo "CRON PERSISTENCE FOUND"
```

Expected: the SHA-256 is recorded as an IOC, the YARA rule matches, and both the planted process and cron entry are detected during the hunt.

Troubleshooting

Symptom	Fix
`yara: rule ... syntax error`	Re-create `eicar.yar` exactly as in step 5; stray quotes break the parser.
`ss: command not found`	Install with `apt-get install -y iproute2`.
EICAR file quarantined	On a host with real AV the file may be deleted; the Killercoda playground has none, so it persists.

Challenge

Add a second YARA rule that matches a byte pattern instead of a string (for example an ELF magic header `{ 7f 45 4c 46 }`), then scan `/bin` and explain why signature matching alone cannot catch a brand-new variant.

Reflection

Objective 2.4 asks you to *analyze indicators*. Which of your three hunt findings (process, cron, port) would survive a reboot, and how does that change which one you investigate first?

Exam tips

- Static analysis never runs the sample; dynamic analysis detonates it in a sandbox.
- SHA-256 is the preferred IOC hash; signature detection is precise but blind to new variants, heuristics catch the unknown with more false positives.
- Linux persistence hides in cron, systemd units, `~/.bashrc` and rc scripts.
- A rootkit hides at kernel/firmware level, so trust the kernel's view less and compare against a known-good baseline.

References

- EICAR Test File — https://www.eicar.org/download-anti-malware-testfile/
- YARA documentation — https://yara.readthedocs.io
- MITRE ATT&CK Persistence (TA0003) — https://attack.mitre.org/tactics/TA0003/
- NIST SP 800-83 Malware Incident Prevention — https://csrc.nist.gov/pubs/sp/800/83/r1/final

---